

nff — power characteristics on ESP32

WiFi/MQTT vs. BLE, per operation — and what it costs a battery

ESP32-WROOM-32E • Rev. A • 2026-07-14 • PRELIMINARY

1 Energy per nff operation

Typical values, 25 °C. Energy at the module's 3.3 V rail, $E = 3.3 \text{ V} \cdot I \cdot t$. BLE column derived from the WiFi column per the derivation in `nff-power-characteristics.tex`. * marks a figure measured on the shunt rig (2026-07-14) and de-embedded to the module's 3.3 V rail; everything else is modelled.

Operation	WiFi / MQTT			BLE			ΔE
	t	I	E	t	I	E	
<code>nff_init()</code> NVS, panic hook	12 ms	40 mA	1.6 mJ	12 ms	40 mA	1.6 mJ	1.00×
Link bring-up * assoc+DHCP / BT init	3.10 s	64.7 mA	662 mJ	0.26 s	62 mA	53 mJ	0.06×
Time sync SNTP / gateway stamps it	1.80 s	95 mA	564 mJ	—	—	0 mJ	gone
Secure session mTLS+MQTT / GATT	1.60 s	110 mA	581 mJ	0.15 s	75 mA	37 mJ	0.06×
Heartbeat, <i>one packet</i> 250 B / 27 B	45 ms	120 mA	17.8 mJ	2.5 ms	120 mA	0.99 mJ	0.06×
Heartbeat, <i>delivered</i> 15 adv events	<i>(requires the session above)</i>			1.50 s	42 mA	208 mJ	note 1
Keepalive MQTT PINGREQ / none	12 ms	115 mA	4.6 mJ	—	—	0 mJ	gone
Command auth ECDSA-P256 verify	55 ms	62 mA	11.3 mJ	55 ms	62 mA	11.3 mJ	1.00×
↔ ping / diag reply	30 ms	118 mA	11.7 mJ	2.5 ms	120 mA	1.0 mJ	0.09×
↔ reboot	0.90 s	88 mA	261 mJ	0.90 s	78 mA	232 mJ	0.89×
Crash report 2.5 kB	180 ms	105 mA	62 mJ	320 ms	90 mA	95 mJ	1.53×
Enrollment announce + cert rollover	3.20 s	112 mA	1.18 J	1.50 s	85 mA	421 mJ	0.36×
Failed-link retry one attempt	1.90 s	108 mA	677 mJ	structurally eliminated			note 2
OTA * 1.2 MB image	9.8 s	150 mA	4.89 J	80 s	108 mA	28.5 J	5.8×
in charge			0.41 mA h			2.40 mA h	note 3
OTA probation soak 5 s	5.00 s	82 mA	1.35 J	5.00 s	55 mA	0.91 J	0.67×
OTA rollback	0.90 s	88 mA	261 mJ	0.90 s	78 mA	232 mJ	0.89×

Note 1 — the window is the cost, not the packet.

One BLE advertisement is 0.99 mJ: *eighteen times cheaper* than an MQTT heartbeat. But a *delivered* heartbeat means holding an advertising window open long enough for the gateway to hear it — 1.5 s, at 208 mJ. **The radio is 7 % of that; the other 93 % is simply having the CPU switched on.** Optimise the window, not the packet.

Note 2 — BLE deletes the SDK's worst battery failure mode.

On WiFi, a device that cannot reach its broker retries *forever* on a ladder that saturates at one attempt per minute (`nff_core.c:31`, hardcoded): 677 mJ each, or **500 mA h per month** — 8 % of a 6 000 mA h battery every month, with no telemetry to show for it. An advertise-only BLE node **has no link to fail**: an unheard advertisement costs exactly what a heard one costs, and nothing more. **This is arguably the strongest reliability argument for BLE here, and it is not a power-efficiency argument at all.**

Note 3 — BLE OTA costs 5.8× more, and it *still* does not matter.

Once the WiFi OTA row was measured * rather than modelled, the penalty grew from 1.29× to **5.8×** — WiFi turned out to be 4.2× faster than assumed (122 kB/s, not 29 kB/s), and BLE has no such headroom. **The conclusion survives anyway**: 2.40 mA h is 0.04 % of a 6 000 mA h battery, and twelve updates a year is 30 mA h — 0.6 % of the annual budget. **Ship the updates.** A 5.8× penalty on 0.6 % is still 0.6 %.

2 What this means for a 6 000 mA h / 12-month node

Reference duty cycle: wake every 5 min, read a sensor, advertise, sleep. Budget: 5 100 mA h usable (85 % derate) over 8 760 h ⇒ **582 μA mean**, or **48.5 μA h per cycle**.

	CPU-active	Sleep floor	I_{mean}	Life
D0 naive, devkit	31.2 s	6.5 mA	10.8 mA	20 days — fails on the <i>sleep floor alone</i>
D1 naive, custom board	31.2 s	20 μA	5.01 mA	6.1 weeks — the literal spec, built the obvious way
D2 short wake	2.5 s	20 μA	380 μA	18.4 months — recommended; no extra parts
D3 30 s window, CPU light-slept	1.2 s	20 μA	394 μA	17.7 months — +32.768 kHz crystal

A 30 s CPU-active window cannot be made to work, on any radio. The duty cycle is 10 %, so the mean budget of 685 μA (un-derated) permits at most 6.85 mA while awake. **No ESP32 executes code at 6.85 mA** — Espressif’s floor is 20 mA to 31 mA (modem sleep, 80 MHz, radio *off*). **Over budget by $\geq 3\times$ with a perfect battery and the radio switched off.**

But “awake for 30 s” and “the CPU runs for 30 s” are different requirements, and they differ by $12\times$ in battery life. D3 keeps the full 30 s *advertising window* and runs the CPU for only 1.2 s of it, the rest in automatic light sleep with the BT controller holding the advertising timing. **Ceiling: the CPU may run for ≤ 3.9 s per 5 min cycle.**

The sleep floor is a board property, not a chip property. A devkit draws **6.5 mA asleep** (AMS1117 5 mA + power LED 1.4 mA + CP2102) — **$11\times$ the entire per-cycle budget, while doing nothing at all.** A custom board with a low- I_q LDO reaches **20 μA** . No firmware recovers a devkit, and any battery-life figure measured on one is meaningless.

The budget, as three numbers

Quantity	Value	Where it comes from
Cycle period T	300 s	Given: wake every 5 min.
Mean-current budget I_{budget}	582 μA	5 100 mA h / 8 760 h (85 % derate).
Charge per cycle	48.5 $\mu\text{A h}$	$I_{\text{budget}} \cdot T$. <i>Everything</i> the device does in a cycle must fit in this.
CPU-active ceiling $t_{\text{on,max}}$	3.9 s	$T \cdot (I_{\text{budget}} - I_{\text{sleep}}) / (I_{\text{awake}} - I_{\text{sleep}})$, with $I_{\text{awake}} = 43.2 \text{ mA}$, $I_{\text{sleep}} = 20 \mu\text{A}$. Every row of the next table is downstream of this one number.

Every nff SDK parameter, and what to set it to

Read from the source, not inferred: `nff-sdk-c/include/nff.h` (all `#defines` are -D-overridable) and `nff-sdk-c/src/nff_core.` Charge figures use the operation costs in §1; * marks those anchored to measurement.

Parameter	Default	Set to	Cost/yr	Why
<i>Tunable today — a -D or an nff_config_t field</i>				
heartbeat_interval_s	30	300	1 577 mA h → 158 mA h	The single most expensive default in the SDK. At 30 s that is 1 051 200 beats/yr at 1.5 μA h each — 26 % of the battery, on keepalives. One beat per wake is the only affordable rate: set it equal to the cycle period.
NFF_OTA_MIN_HEALTHY_MS	5000	1000	negligible	Not about energy — about the ceiling. A 5 s soak holds the CPU up for longer than the entire 3.9 s budget , so a post-OTA cycle cannot close. 1 s still catches a boot loop.
NFF_OTA_CONFIRM_TIMEOUT_S	60	60 keep	0	Runs only after an OTA (≈12/yr) and only until the broker reconnects. Free.
NFF_OTA_MAX_TRIAL_BOOTS	3	3 keep	0	Crash-loop guard. Costs nothing; removing it risks a brick.
NFF_OTA_MAX_DL_RETRIES	4	4 keep	< 0.1 mA h	Worst case 4 full downloads = 20 J*, once. Irrelevant against 6 000 mA h.
NFF_OTA_DL_RETRY_BACKOFF_MS	2000	2000 keep	0	Radio idle while backing off.
NFF_OTA_MIN_HEAP_FLOOR	20 000	keep	0	RAM gate, not energy.
NFF_MQTT_BUFFER_SIZE	1024	keep	0	RAM. (Never below 1024 — OTA commands are ≈400 B and PubSub-Client’s 256 B default silently drops them.)
NFF_TIMESTAMP_WINDOW_S	300	keep	0	Replay window. Do not widen it to dodge Sntp — that is a security control, not a power knob.
NFF_LOG_LINES / _LEN	32 / 128	keep	0	RAM.
NFF_NONCE_RING_SIZE	16	3 keep	0	RAM.

Read the table in order. Only *one* tunable parameter — `heartbeat_interval_s` — moves the needle, and it saves 1 419 mA h/yr. **Every other lever that matters is in the bottom half of the table, and none of them is a setting.** Two are hardcoded constants and two are features that do not exist. **A 6 000 mA h / 12-month node cannot be configured out of the current SDK; it has to be built.** That is the finding, and it is why the SDK work list specifies work rather than settings.